



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza

Memoria de Práctica 3: Búsqueda con retroceso

Asignatura: Algoritmia Básica (Curso 2025/26)

Autores:

Imad Habib Jali

Jorge Bellido Lobera

901421@unizar.es

903080@unizar.es

1. Diseño del Algoritmo y Espacio de Estados

El problema requiere determinar la ubicación de k nuevos centros de urgencias en una región con n localidades para minimizar el tiempo máximo de respuesta ante emergencias. Este es un problema de optimización de tipo minimax.

- **Representación de la solución:** Una solución candidata se representa como una tupla $S = (s_1, s_2, \dots, s_k)$ donde cada s_i es el índice de la localidad elegida para albergar un nuevo centro.
- **Árbol del espacio de soluciones:** Se ha modelado como un árbol n -ario de profundidad exacta k . Para evitar la explosión combinatoria provocada por permutaciones equivalentes (ej. evaluar la solución $\{1, 3\}$ y posteriormente la $\{3, 1\}$), se ha impuesto una restricción de orden: $s_1 < s_2 < \dots < s_k$. Gracias a la restricción de orden creciente, el tamaño del espacio de búsqueda teórico no es una variación, sino una combinación. El tamaño total del árbol de búsqueda (incluyendo la raíz nula y los nodos internos) viene dado por la suma de combinaciones desde $i=0$ hasta k de $C(n, i)$.

2. Decisiones de Implementación

Para garantizar un rendimiento óptimo acorde a los requisitos de la práctica, la arquitectura del software se ha dividido en dos fases fundamentales:

Precálculo (Algoritmo de Floyd-Warshall):

Antes de iniciar el Backtracking, se ejecuta Floyd-Warshall para calcular la matriz de distancias mínimas entre todos los pares de nodos en un tiempo $O(n^3)$.

En cada nodo del árbol de búsqueda es imperativo evaluar la función de coste (tiempo máximo de acceso). Si utilizáramos algoritmos como Dijkstra desde cero en cada estado del Backtracking, la complejidad temporal haría inviable el cálculo. Al tener las distancias precalculadas en memoria, la evaluación de la cobertura sanitaria en cada nodo del árbol requiere únicamente tiempo $O(n)$.

Lógica de Poda:

- **Poda Combinatoria:** Si en un nivel dado el número de localidades candidatas restantes es menor que el número de centros que quedan por asignar, la rama se poda inmediatamente.
- **Poda por Optimalidad Absoluta:** Si se encuentra una solución con coste 0 (todas las localidades tienen un centro), se detiene la exploración por completo al haber alcanzado el límite inferior teórico.
- **Poda por Cota Inferior:** Se precalcu la matriz de sufijos que indica la distancia mínima desde cada nodo u hacia cualquier subconjunto restante de candidatos. En cualquier nodo interno del árbol, comprobamos si el mínimo entre el acceso actual y la distancia mínima a los restantes es mayor o igual al mejor coste encontrado. De ser así, se poda la rama, ya que es matemáticamente imposible que mejore el óptimo temporal encontrado.

3. Experimentación y Análisis de Resultados

Se han llevado a cabo pruebas incrementales para verificar la corrección funcional y evaluar el impacto paramétrico de n y k .

Caso de Prueba	n	k	T. Ejecución (ms)	Nodos Generados	Valor Óptimo	Solución
1. Estándar	5	1	0.001	5	10	5
2. Estándar	6	2	0.001	8	3	1, 3
3. Estrella	6	1	0.000	6	10	6
4. Lineal	8	2	0.002	23	10	4, 7
5. Saturación	4	3	0.001	4	0	1, 3, 4

Automatización y Análisis de Pruebas Masivas

Para llevar el algoritmo al límite y facilitar el análisis de complejidad, hemos desarrollado un script en Python (`generadorPruebas.py`) diseñado para generar casos de prueba masivos y aleatorios con grafos conexos. Asimismo, se ha implementado un script en Bash (`ejecutar.sh`) que automatiza la compilación con optimización `-O3`, la generación de datos y la ejecución de todos los casos.

Análisis de Rendimiento:

- **Corrección:** En los casos base, el algoritmo resuelve el problema en tiempos sub-milisegundo, demostrando que la poda por optimalidad y cota inferior dirigen rápidamente la búsqueda hacia soluciones óptimas, descartando ramas inútiles casi de inmediato en grafos pequeños.
- **Comportamiento ante la Saturación:** Cuando el número de centros a instalar (k) es muy cercano al número de localidades (n), como se observa en el Caso 5, la poda combinatoria y absoluta (coste 0) entran en acción, resolviendo el problema explorando apenas 4 nodos en total.
- **Crecimiento Exponencial (Casos Python):** El análisis general de las salidas generadas por el script masivo confirma la naturaleza teórica del problema. A medida que incrementamos las dimensiones del grafo a decenas de localidades y múltiples centros, tanto los tiempos de ejecución como los nodos explorados sufren un crecimiento exponencial. Sin embargo, la matriz de cota inferior precalculada permite que el algoritmo siga siendo funcional y ofrezca respuestas en rangos de tiempo viables (de fracciones de segundo a unos pocos segundos), demostrando que la cota es admisible y altamente restrictiva.

4. Conclusiones

La combinación de la programación dinámica (Floyd-Warshall) como soporte para un algoritmo de búsqueda exhaustiva (Backtracking) resulta en un diseño de software escalable. La imposición del orden lexicográfico y las podas por combinatoria han permitido transformar un problema exponencialmente complejo en uno manejable para instancias de tamaño medio, garantizando siempre la obtención del tiempo de respuesta mínimo para la región de Sierra de las Dehesas. Además, la creación de un entorno de automatización de pruebas nos ha permitido validar empíricamente que, aunque el coste asintótico dominante sigue siendo exponencial, las podas implementadas reducen drásticamente la base matemática de dicha explosión combinatoria.